



Bài 3 Cấu trúc dữ liệu và giải thuật 2

Công nghệ thông tin (Đại học Xây dựng Hà Nội)



Scan to open on Studocu

Chương 3. CÂY – TREE

3.1. Khái niệm kiểu dữ liệu Tree

Cấu trúc Tree cho phép tận dụng ưu điểm của cả 2 cấu trúc dữ liệu tuyến tính và cấu trúc dữ liệu liên kết. Do vậy, cấu trúc Tree được sử dụng rất nhiều trong các hệ quản trị cơ sở dữ liệu để hỗ trợ cho việc đánh chỉ mục, giúp cho thao tác truy vấn dữ liệu được nhanh hơn.

Cấu trúc dữ liệu kiểu Tree là tập hợp các nút (hay gọi là đỉnh) và những cạnh nối với các cặp nút với nhau.

- Nút gốc (○) không có cạnh nào đến nó, chỉ có từ nó đi ra

- Nút con - nút đi ra từ nút gốc

- Lá không có cạnh nào từ nó đi ra.

Ví dụ 1 — cho cây :

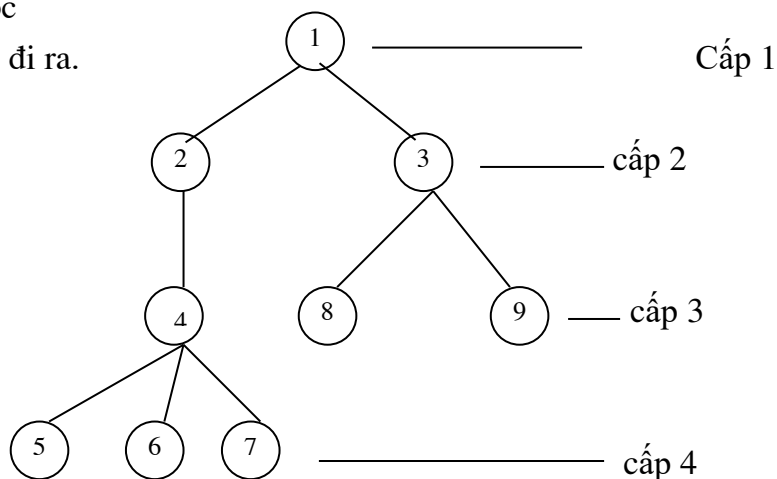
Nút 1 là gốc

nút 2,3 là nút con và 1 là nút bố

nút 2,3 còn gọi là nút anh em

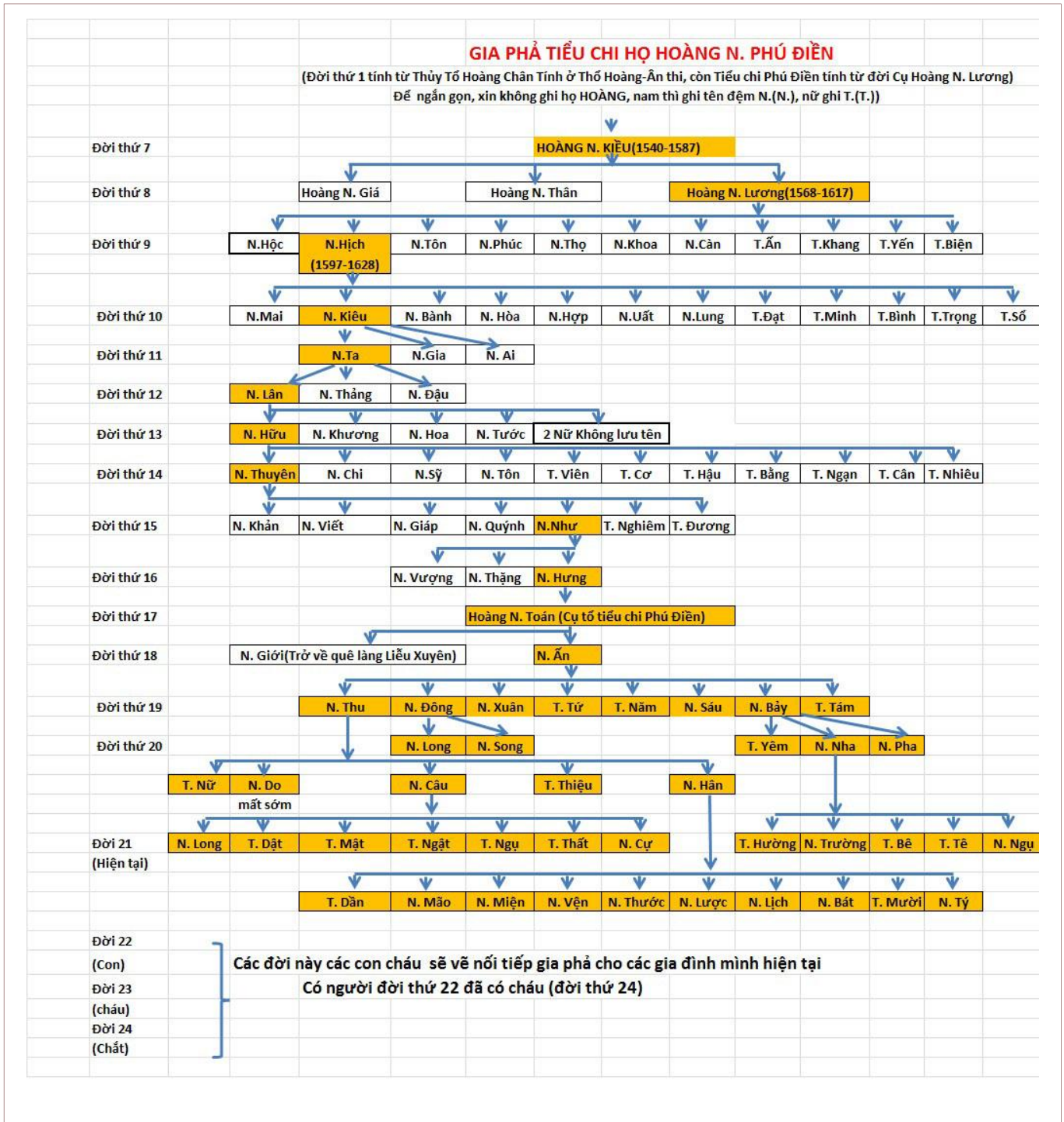
nút 5,6,7,8,9 gọi là lá

Hình 1.



Ví dụ 2 : cây gia hệ

Gia phả của một dòng họ là một cấu trúc dạng cây :



Hình 2.

Một số thuật ngữ và định nghĩa về cấu trúc CÂY:

Các cấp (các mức) của cây

Cây được biểu diễn theo dạng phân cấp, có các mức được đánh số từ 1,2...theo hướng xuống dưới, cấp độ của một nút là cơ sở để tính **độ dài của đường đi** từ nút gốc tới nút đó.

Độ dài của đường đi của một nút được tính bằng số cạnh đi từ nút gốc tới nút đó. Trong hình vẽ trên, các nút ở cấp 2 có độ dài bằng 1, ở cấp 3 có độ dài bằng 2, ở cấp 4 có độ dài bằng 3.

Chiều cao của cây

Chiều cao của cây là **đường đi dài nhất** đi từ nút gốc tới nút lá xa nhất của cây.

Trong cây trên thì chiều cao của cây là 3, là đường đi dài nhất, từ nút gốc tới các nút lá số 5, 6, 7.

Các loại cây

Có nhiều cách phân loại cây. Chỉ tiêu phân loại quan trọng nhất là số nút cực đại của các nút con bất kỳ mà trong cây có, số cực đại này được gọi là **order** (bậc/xếp loại/kiểu) -của cây.

Dựa theo order này người ta phân ra 3 loại cấu trúc cây:

- ❖ Cây tổng quát – là cấu trúc cây không giới hạn về order ;
- ❖ Cây $n_nút$ – là cấu trúc cây mà mỗi nút có không quá n nút con;
- ❖ Cây nhị phân – là cấu trúc cây mà mỗi nút có không quá 2 nút con.

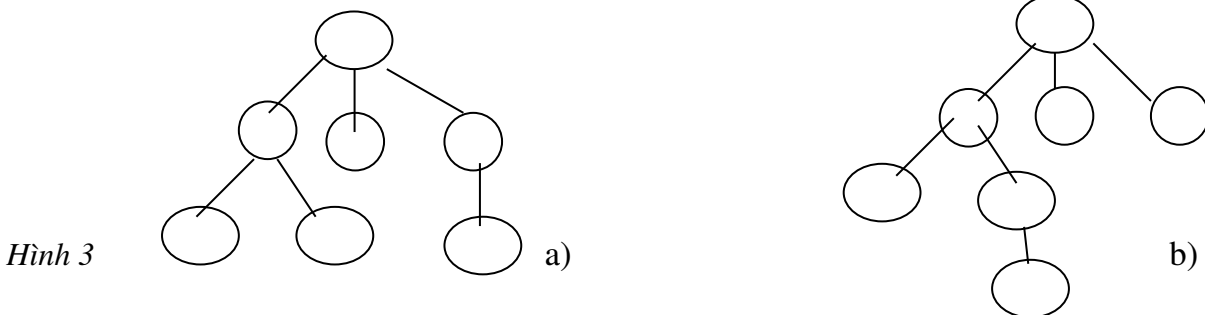
Cây cân bằng -Balanced tree

Một cách khác để phân loại cây là độ cân bằng. Có nhiều định nghĩa khác nhau về cây cân bằng, phụ thuộc vào thuật toán được sử dụng . Nói chung, một cây được coi là cân bằng khi :

- ✓ Tất cả các lá của cây đều ở trên cùng một cấp hoặc trong cùng một cấp của nhau;
- ✓ Chiều cao của cây $\leq \log_2 m$, trong đó m là số nút trong cây, n là order của cây.
- ✓ Hệ số cân bằng của tất cả các nút trong cây $\leq \pm 1$.

Khi order = 2, gọi là cây nhị phân và chiều cao của cây có N nút sẽ là $\log_2 N$.

Ví dụ về cây cân bằng (bên trái) và cây không cân bằng (bên phải, vì chiều cao cây $= 3 > \log_2 7$):



Cây hoàn thiện (complete tree)

Khái niệm cây hoàn thiện liên quan đến cây cân bằng.

Một cây được coi là **hoàn thiện** nếu nó cân bằng và tất cả các lá ở cấp dưới cùng đều ở phía bên trái của cây.

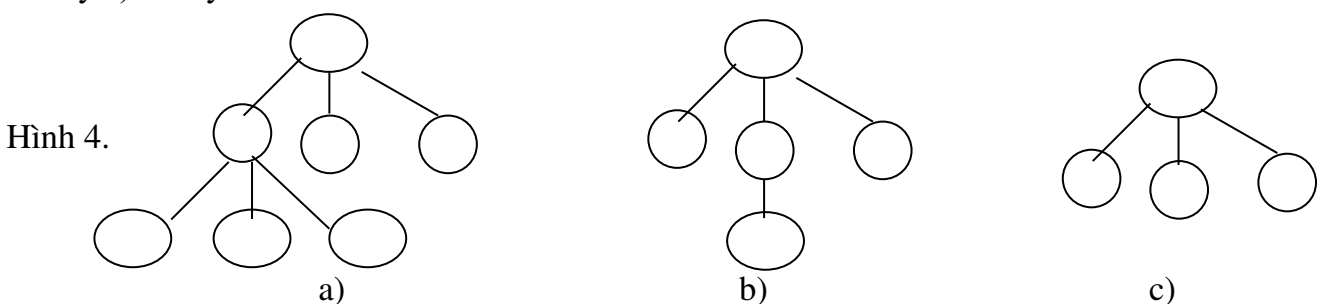
Định nghĩa này có ý nghĩa đối với việc tổ chức dữ liệu cây trong những cài đặt cụ thể.

Có thể nói cách khác, một cây nhị phân hoàn thiện có 2^k nút ở mỗi cấp thứ k , trừ cấp cuối cùng các nút chỉ ở bên trái nhất.

Cây cân bằng bên trái ở hình trên chưa phải là cây hoàn thiện

Cây đầy đủ (full tree)

Một cây n -nút là cây **đầy đủ** nếu tất cả các lá của cây đều ở trong cùng một cấp và mỗi nút hoặc là lá hoặc có đúng n nút con. Trong các cây 3_nút dưới đây, cây a) và c) là hoàn thiện, nhưng chỉ có cây c) là đầy đủ.



3.2. Khái quát các cách cài đặt Cây

Có một vài cách cài đặt cây.

Cách cài đặt phổ biến nhất là dùng cấu trúc liên kết. Mỗi nút được coi là một lớp class `TreeNode` tương tự như lớp `Node` trong danh sách liên kết. Giống như danh sách liên kết đôi, mỗi nút có con trỏ tới nút con trái, nút con phải; khi với tư cách là nút con thì nó đang được nút cha trỏ tới.

Một cách cài đặt khác là kiểu *cây đệ quy* có sử dụng các liên kết. Cách này sẽ liên quan đến việc xác định mỗi nút là một cây với các thuộc tính cho các nút con của nó cũng là cây. Vì vậy, mỗi nút cùng với tất cả các nút con ở các cấp của nó cũng là một cây.

Tuy cây là một cấu trúc phi tuyến, nhưng có thể dùng mảng để lưu trữ dữ liệu các nút của cây.

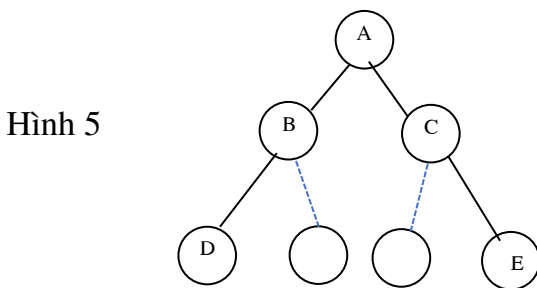
Khi sử dụng mảng để lưu trữ dữ liệu các nút của cây nhị phân, có 2 cách tiếp cận :

a) Cách 1 : coi cây là cây hoàn thiện, bất kỳ một nút cha nào cũng có 2 nút con, trừ các nút lá.

Như vậy, đối với nút được lưu trữ ở vị trí thứ i của mảng thì nút con trái được lưu ở vị trí $2*i$, nút con phải ở vị trí $2*i + 1$.

Cách tiếp cận này có hiệu quả trong việc quản lý dung lượng bộ nhớ, tuy nhiên đối với cây không hoàn thiện sẽ lãng phí bộ nhớ vì có những nút không tồn tại, ô nhớ bị bỏ trống.

Ví dụ cho cây :



Lập mảng :

i	1	2	3	4	5	6	7
T[i]	A	B	C	D			E

b) Cách tiếp cận thứ 2 là cài đặt cây theo cách quản lý của Hệ điều hành, chỉ những nút nào có mặt, xuất hiện trước được lưu trước, liên kết, không bỏ trống ô nhớ, mỗi nút được lưu trữ theo chỉ số mảng, theo thứ tự nút được nhập lúc tạo cây.

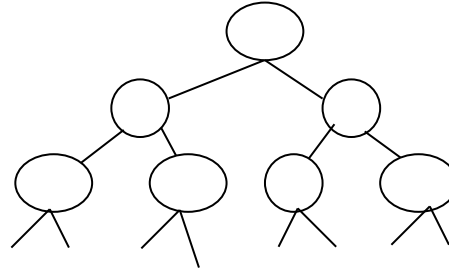
Với cây trên thì mảng $T[]$ có dạng :

i	1	2	3	4	5
T[i]	A	B	C	D	E

Rõ ràng cách này tiết kiệm ô nhớ hơn nhiều. Tuy nhiên khi thêm bớt nút trong cây, cách này phiền toái thêm việc phải dịch chuyển vị trí các ô nhớ để duy trì sự liên kết.

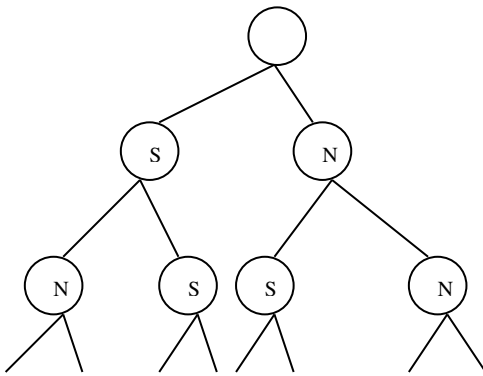
3.3.Cây nhị phân

3.3.1.Khái niệm. Cây nhị phân là cây trong đó mỗi nút có nhiều nhất hai nút con đi ra.

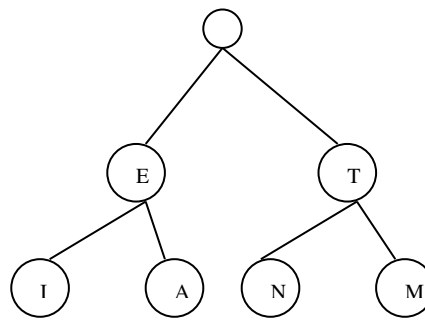


Hình 6

Cây nhị phân là công cụ mô phỏng tốt các quá trình có 2 khả năng xảy ra: tung hứng đồng tiền (S- sấp, N - ngửa) và ký hiệu morse:

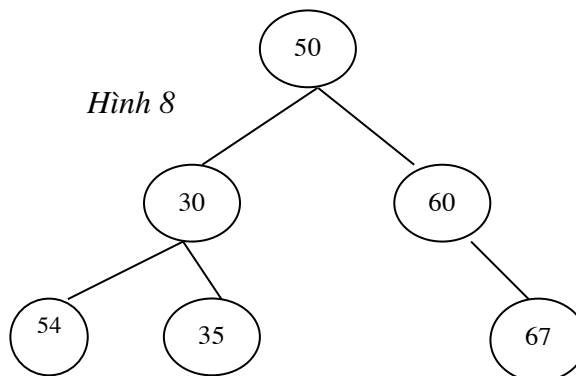


Hình 7



Ta có thể truy nhập đến một nút bất kỳ của cây nhị phân nếu bảo trì một con trỏ chỉ đến nút gốc của cây. Dùng cấu trúc để biểu diễn các nút và các con trỏ.

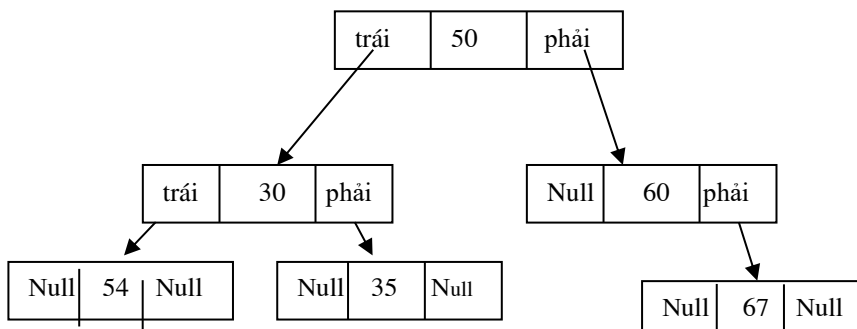
Ví dụ: cây nhị phân



Hình 8

Số nút tối đa $= 2^3 - 1 = 7$

Có thể được biểu diễn



Một số hàm thường dùng trên cấu trúc cây nhị phân:

- + Tạo cây rỗng
- + Kiểm tra cây rỗng không
- + Tạo cây với các dữ liệu mới
- + Các kiểu duyệt cây
- + Xác định nút con trái của một nút
- + Xác định nút con phải của một nút
- + Kiểm tra có phải là nút lá không
- + Xác định số nút của cây
- + Tìm một nút(đỉnh) theo dữ liệu của nút trên cây
- + Thêm nút/xóa nút
- + Hiện cây

Có hai cách thường dùng để biểu diễn cây nhị phân:

- Biểu diễn vị trí và dữ liệu các nút bằng mảng
- Biểu diễn cây bằng con trỏ

3.3.2. Định nghĩa đệ quy cây nhị phân

Một cây nhị phân :

- Hoặc là rỗng (phần neo)
hoặc
- Chứa một nút gọi là gốc, nó có hai con trỏ chỉ đến hai cây con nhị phân bên trái, bên phải.

3.4. Số lượng nút của một cây nhị phân

Ta có thể xác định số lượng tối đa các nút của một cây nhị phân (gốc ở mức $i=1$) :

1. Số lượng tối đa các nút ở mức i của cây nhị phân là 2^{i-1} với $i \geq 1$ và
2. Số lượng tối đa các nút trong cả cây nhị phân có k mức là $2^k - 1$, $k \geq 1$.

Ví dụ : cây có 3 mức, thì ở mức $i=3$, có tối đa $2^2 = 4$ nút, Số lượng tối đa các nút trong cả cây là $2^3 - 1 = 7$

Chứng minh quy nạp

(1) :

- Gốc ở mức $i=1$ như vậy $2^{i-1} = 2^0 = 1$ là đúng !
- Giả sử kết quả trên đúng với $\forall j, 1 \leq j < i$, số lượng các nút ở mức j là 2^{j-1} .

Ta cần chứng minh điều này đúng cho $i = j+1$.

ở mức $i-1$, theo giả thiết là có 2^{i-2} nút, như vậy, cứ mỗi nút ở mức $i-1$ này lại có tối đa là 2 nút ở mức i tức là số nút ở mức i tối đa bằng 2 lần số nút ở $i-1$:

$$2^{i-2} \times 2 = 2^{i-1}$$

(2). Để tính số nút tối đa trong cây k mức ta lấy

$$\sum_{i=1}^k 2^{i-1} = 2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{i-1} = 2^k - 1$$

Chuỗi $1+a+a^2+\dots+a^n + \dots$ khi $a=2$ có tổng là $\frac{1-2^k}{1-2}$

$$\text{vì } S_n = \frac{1 - a^{n+1}}{1 - a} \quad \text{khi } a \neq 1$$

Trong ví dụ trên với $k=6$ ta có số nút tối đa là $2^6-1=63$

- Nếu một *cây nhị phân đầy đủ* mức k được gọi là cây nhị phân k -mức có 2^k-1 nút. (Ngược lại, ta thấy, một cây nhị phân đầy đủ N nút thì có mức là $\lceil \log_2 N \rceil + 1$). Trong cây nhị phân đầy đủ các nút được đánh số theo trình tự từ trên xuống dưới, từ trái qua phải. Cách đánh số này cho ta định nghĩa một cây nhị phân hoàn thiện. Một cây nhị phân N nút với k mức là hoàn thiện khi và chỉ khi các nút của nó tương đương với việc đánh số từ 1 đến n trong cây nhị phân đầy đủ mức k . Chúng ta có thể lưu các nút trong mảng *tree* một chiều, nút thứ i sẽ lưu trong phần tử *tree[i]*. *(nên thêm sơ đồ 1 cây hoàn thiện và lưu nó trong mảng tree)*

Ta có bổ đề sau :

Nếu một cây nhị phân hoàn thiện N nút được biểu diễn tuần tự như trên thì với một nút i bất kỳ ta có :

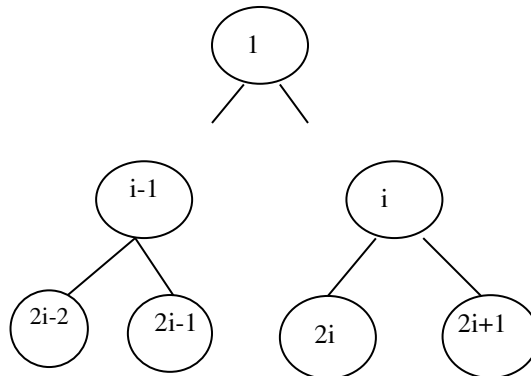
- (1) nút bố của i sẽ ở vị trí $(i/2)$ với $i \neq 1$. Khi $i=1$, i sẽ là gốc và không có nút bố.
- (2) nút con trái của i sẽ ở vị trí $2i$ nếu $2i \leq n$. Nếu $2i > n$ thì i không có nút con trái.
- (3) nút con phải của i sẽ ở vị trí $2i+1$ nếu $2i+1 \leq n$. Nếu $2i+1 > n$ thì i không có nút con phải.

Chứng minh :

Ta chứng minh cho (2), còn (3) sẽ suy ra từ (2) và từ việc đánh số nút trên cùng một mức từ trái sang phải. (1) sẽ suy ra từ (2) và (2).

Chứng minh theo quy nạp : với $i = 1$, nút con trái rõ ràng là bằng 2, nếu số nút của cây $n=1$, khi đó $2*(i=1) > n$ thì $i=1$ không có nút con trái.

Giả thiết là nút con trái của j ở vị trí $2j$ đã đúng với mọi j từ 1 đến $i-1$, ta cần khẳng định điều này cũng đúng với i .



Khi đó hai nút con của $(i-1)$ đã là $2i-2$ và $2i-1$, thì nút tiếp theo là nút con trái của i phải là $2i$ và nút con phải của i phải là $2i+1$. Nếu $2i > n$ thì i không có nút con trái, nếu $2i+1 > n$ thì i sẽ không có nút con phải. Ta đã chứng minh cho cả 3 phần của bổ đề.

3.5. Cài đặt cây nhị phân

3.5.1. Khai báo cấu trúc nút cho cây nhị phân:

```
struct TNode
{
    int du_lieu ;
    TNode *Lchild, *Rchild ;
}
```

3.5.2. Khai báo cây nhị phân

```
typedef TNode *Tree ;
```

3.5.3. Tạo nút cây nhị phân

Khi nhập dữ liệu hay thực hiện các thao tác trước khi thêm nút, cần phải tạo một nút để nhận dữ liệu mới. Nút mới tạo sẽ có dữ liệu, còn con trỏ trái, phải đều chưa trỏ vào đâu nên được gán NULL.

```
TNode* TaoNode(TNode* node, int du_lieu)
{
    node = new TNode;
    node->du_lieu = du_lieu;
    node->Lchild = NULL;
    node->Rchild = NULL;
    return node;
}
```

3.5.4. Tạo dựng cây nhị phân

Để tạo cây phải nhập dữ liệu từ gốc, và dữ liệu gốc phải khác 0, sau đó gọi đệ quy nhập cây con bên trái; gọi đệ quy nhập cây con bên phải.

```
TNode* TaoTree(TNode* node, int du_lieu)
{
    cout<<"\n    Nhap du lieu cho nut : "; cin>>du_lieu;
    if (du_lieu == 0) return NULL;
    node = TaoNode(node,du_lieu);
    cout<<"    Nhap Du lieu node con TRAI cua nut "<<du_lieu<<" : ";
    node->Lchild = TaoTree(node->Lchild,du_lieu);
    cout<<"    Nhap Du lieu node con PHAI cua nut "<<du_lieu<<" : ";
    node->Rchild = TaoTree(node->Rchild,du_lieu);
    return node;
}
```

Chú ý: khi đến nút lá, không có nút con thì nhập Du lieu node con TRAI =0 và Du lieu node con PHAI = 0 để chuyển về nút cha của nút hiện thời.

3.6. Duyệt cây nhị phân

Việc duyệt cây nhị phân theo đệ quy có ba thao tác cơ bản:

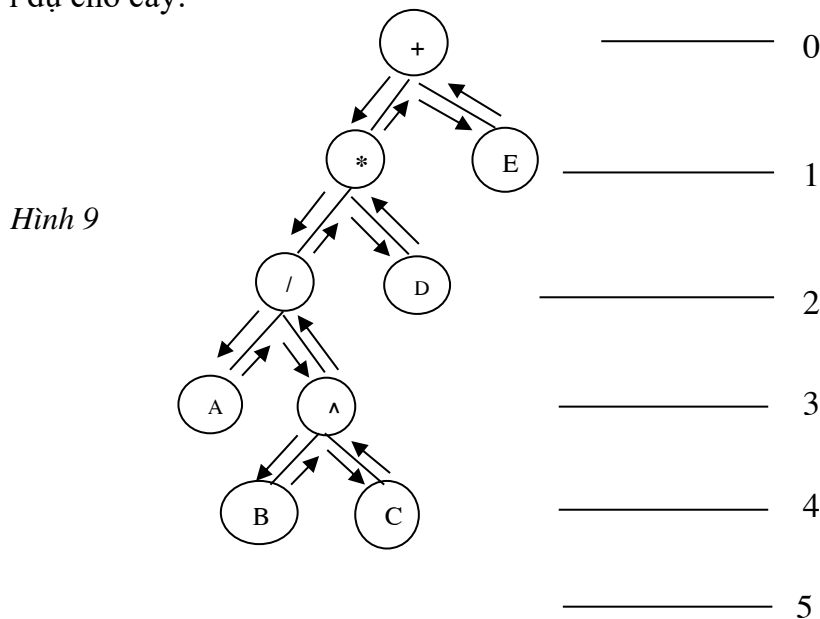
- 1) Xử lý nút gốc (D- Data - Du_Lieu) cụ thể là hiện dữ liệu n
- 2) Duyệt cây con bên trái (L- Left)
- 3) Duyệt cây con bên phải (R- Right)

Nếu duyệt theo trình tự 1-2-3 là đầu tiên xử lý dữ liệu(D), sau đó duyệt cây con bên trái(L), rồi duyệt cây con bên phải(R), ký hiệu là DLR và có tên gọi là kiểu duyệt tiền tự - PreOrder. Lần lượt hoán vị 3 thao tác trên ta có 6 thứ tự duyệt :

LDR,LRD,DLR,DRL,RDL,RLD

Nếu ta chỉ thừa nhận chỉ duyệt từ trái sang phải thì chỉ có ba thứ tự duyệt chính là: LDR, LRD, DLR với thường được gọi là duyệt trung tự(InOrder), hậu tự(PostOrder), tiền tự(PreOrder).

Ví dụ cho cây:



Duyệt theo LRD sẽ cho kết quả là $ABC^{\wedge}/D*E+$

Ta sẽ xem kết quả ba kiểu duyệt nút LDR,LRD,DLR .

3.6.1.Duyệt theo LDR - Inorder

Với kiểu duyệt này, ta đi xuống bên trái(L) của cây cho đến khi không đi tiếp được nữa. Sau đó "thăm" nút, chuyển lên "thăm" nút gốc con(D), sau đó xuống bên phải(R) và tiếp tục, nếu không chuyển được xuống phải nữa thì "thăm" nút rồi quay trở lên một nút.

<u>Goi Inorder</u>	<u>Giá trị ở gốc</u>	<u>Kết quả</u>
gốc chính	+	
1	*	
2	/	
3	A	
4	NULL	viết 'A'
3	/	viết '/'
3	^	
4	B	
5	NULL	B
4	^	^
4	C	
5	NULL	C
1	*	*
2	D	
3	NULL	D
0	+	+
1	E	
2	NULL	E
0	Exit	

Kết quả :

A/B^C*D+E

Ta có hàm duyệt Inorder :

```
void InOrder(Tree GOC)// LDR
{ if(GOC!=NULL)
  {InOrder(GOC->Lchild);
   cout<<"-"<<GOC->du_lieu;//duyet goc
   InOrder(GOC->Rchild);
  }
}
```

3.6.2. Duyệt theo DLR - PreOrder - xử lý dữ liệu trước- tiên tự

Với kiểu duyệt này, ta xử lý dữ liệu nút gốc, sau đó đi xuống bên trái của cây, xử lý dữ liệu các nút gốc con cho đến khi không đi tiếp được nữa., chuyển lên, bỏ qua nút gốc con đó xử lý dữ liệu cho đến khi gặp cây con bên phải, xuống cây con bên phải và tiếp tục... cho đến khi về nút gốc .

Ta có hàm duyệt PreOrder - DLR:

```
void PreOrder(Tree GOC)//duyet theo thu tu truooc
{ if(GOC!=NULL)
  {cout<<"-"<<GOC->du_lieu;
   PreOrder(GOC->Lchild);
   PreOrder(GOC->Rchild);
  }
}
```

<u>Goi Preorder</u>	<u>Giá trị ở gốc</u>	<u>Kết quả</u>
gốc chính	+	viết '+'
1	*	viết '*'
2	/	viết '/'
3	A	
4	NULL	viết 'A'
3	/	viết '/'
3	^	viết '^'
4	B	
5	NULL	B
4	^	
4	C	
5	NULL	C
1	*	
2	D	
3	NULL	D
0	+	

1	E	
2	NULL	E
0	Exit	

Kết quả sẽ cho $+*/A^BCDE$

3.6.3. Duyệt theo LRD - PostOrder- hậu tự

Với kiểu duyệt này, từ nút gốc đi xuống bên trái của cây cho đến khi không đi tiếp được nữa, **xử lý dữ liệu** nút lá rồi chuyển lên, sau đó xuống cây con bên phải đi cho đến khi không đi tiếp được nữa, **xử lý dữ liệu** nút lá rồi chuyển lên, sau đó xuống cây con bên phải đi cho đến khi không đi tiếp được nữa, **xử lý dữ liệu** nút lá rồi chuyển lên ... tiếp tục cho đến khi về nút gốc

Ta có hàm duyệt PostOrder :

```
void PostOrder(Tree GOC)//duyet theo thu tu sau LRD
{ if(GOC!=NULL)
  { PostOrder(GOC->Lchild);
    PostOrder(GOC->Rchild);
    cout<<" "<<GOC->du_lieu;
  }
}
```

<u>Gọi PostOrder</u>	<u>Giá trị ở gốc</u>	<u>Kết quả</u>
gốc chính	+	
1	*	
2	/	
3	A	
4	NULL	A
3	/	
3	^	
4	B	
5	NULL	B
4	^	
4	C	
5	NULL	C
3	^	^
2	/	/
1	*	
2	D	
3	NULL	D
1	*	*
0	+	
1	E	
2	NULL	E
0	+	+

Kết quả :

$ABC^{\wedge}/D*E+$

3.7. Trả về con trái, trả về con phải, kiểm tra nút lá, đếm số nút, tìm nút

3.7.1. Hàm trả về con trái

```
TNode * Lchild(Tree GOC){
    if(GOC!=NULL) return GOC->Lchild;
    else return NULL;
}
```

3.7.2. Hàm trả về con phải

```
TNode * Rchild(Tree GOC)
{ if(GOC!=NULL) return GOC->Rchild;
  else return NULL;
}
```

3.7.3. Hàm kiểm tra nút lá

```
int LA(Tree GOC) //– kiểm tra nút lá
{ if(GOC!= NULL)
    return (GOC->Lchild==NULL && GOC->Rchild==NULL ) ;
  else
    return 0;
}
```

3.7.4. Hàm đếm số nút trên cây

```
int DemNode(Tree GOC) //Đếm số nút trong cây
{ if(GOC==NULL) return 0;
  else return (1+DemNode(Lchild(GOC))+ DemNode(Rchild(GOC)));
}
```

3.7.5. Tìm kiếm nút theo dữ liệu nút

Hàm này làm nhiệm vụ tìm nút có dữ liệu bằng dữ liệu nhập từ bàn phím.

Nếu tìm thấy thì trả về nút, nếu không thì trả về NULL.

Khai báo Tree tg (nút trung gian). Gọi đệ quy tìm con trái. Nếu con trái không có, tìm con phải.

```
TNode* TimNode(Tree GOC, int du_lieu) // tìm nút theo du_lieu
{ if(GOC!=NULL)
    {
        if (GOC->du_lieu == du_lieu) return GOC;
        TimNode(GOC->Lchild,du_lieu);
        TimNode(GOC->Rchild,du_lieu);
    }
}
```

Bài tập về nhà.

1. Thực hiện lại bằng tay các kiểu duyệt cây LDR, DLR, LRD đối với cây ở hình 9.
2. Gõ nhập lại chương trình, sửa kiểu dữ liệu nút là char để chạy cho cây ở hình 9, so sánh kiểm tra xem kết quả chạy chương trình so với kết quả làm bằng tay ở mục bài tập 1.

Hướng dẫn tạo chương trình BTree

Chương trình cần có các phần :

- Các dòng header

```
#include <iostream>
#include <iomanip>
using namespace std;
```
 - Khai báo cấu trúc struct Tnode

```
typedef TNode *Tree;
```
 - Hàm TNode* TaoNode(TNode *node, int du_lieu)
 - Hàm TNode* TaoTree(TNode *node, int du_lieu) // tạo cây
 - Hàm Duyệt theo InOrder - thu tu giu
 - Hàm PreOrder(Tree GOC)//duyet theo thu tu truo
 - Hàm PostOrder(Tree GOC)//duyet theo thu tu sau
 - Hàm tìm nút theo du_lieu, theo kiểu duyệt InOrder

```
TNode* TimNode(Tree GOC, int du_lieu)
```
 - Hàm TNode * Lchild(Tree GOC)
 - Hàm TNode * Rchild(Tree GOC)
 - Hàm int LA(Tree GOC) //- kiểm tra nút lá
 - Hàm int DemNode(Tree GOC) //Đếm số nút trong cây
 - Hàm int main()
- ```
{ Tree GOC;
 GOC = NULL; //Tao cay rong
 TNode *node=NULL;
 int du_lieu;

```

Gọi hàm tạo cây : GOC = TaoTree(node,du\_lieu); (chú ý hiện lời nhắc

- “Nhập số 0 để chuyển sang nhập node khác hoặc thoát”

Gọi các hàm duyệt cây theo các kiểu ;

Gọi hàm đếm Số nút của cây;

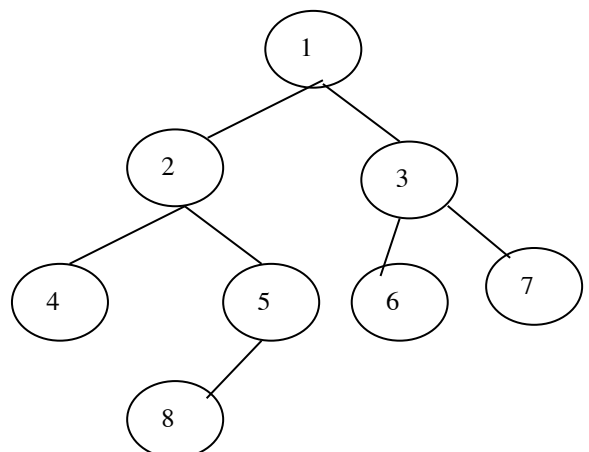
Gọi hàm “ Tìm kiếm nút theo du\_lieu ”;

system("pause");

return 0;

}

Hãy chạy chương trình cho cây :



Kết quả của Chương trình  
ví dụ đã chạy để tham khảo :

```
Nhap so 0 de chuyen sang nhap node khac hoac thoat
Nhap du lieu cho nut : 1
Nhap Du lieu node con TRAI cua nut 1 :
Nhap du lieu cho nut : 2
Nhap Du lieu node con TRAI cua nut 2 :
Nhap du lieu cho nut : 4
Nhap Du lieu node con TRAI cua nut 4 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 4 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 2 :
Nhap du lieu cho nut : 5
Nhap Du lieu node con TRAI cua nut 5 :
Nhap du lieu cho nut : 8
Nhap Du lieu node con TRAI cua nut 8 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 8 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 5 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 1 :
Nhap du lieu cho nut : 3
Nhap Du lieu node con TRAI cua nut 3 :
Nhap du lieu cho nut : 6
Nhap Du lieu node con TRAI cua nut 6 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 6 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 3 :
Nhap du lieu cho nut : 7
Nhap Du lieu node con TRAI cua nut 7 :
Nhap du lieu cho nut : 0
Nhap Du lieu node con PHAI cua nut 7 :
Nhap du lieu cho nut : 0

So nut cua cay hien co : 8

Duyet cay theo thu tu giua InOrder :
-4-2-8-5-1-6-3-7

Duyet cay theo thu tu truoc PreOrder :
-1-2-4-5-8-3-6-7

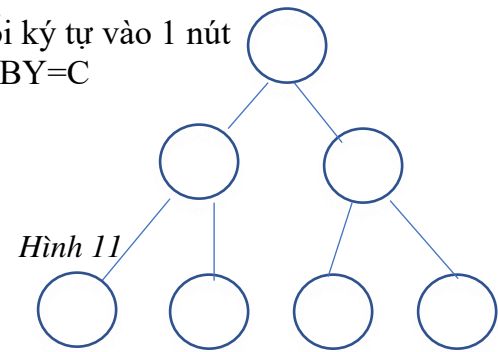
Duyet cay theo thu tu sau PostOrder :
-4-8-5-2-6-7-3-1

Tim kiem nut theo du lieu
Nhap du lieu nut can tim: 3

Da Tim thay nut co du lieu = 3Press any key to conti
```

**Bài tập :**

1) Có 7 ký tự +, -, A, B, C, X, Y, hãy vẽ cây như hình 11 và viết mỗi ký tự vào 1 nút của cây nhị phân đó để khi duyệt theo LRD sẽ cho biểu thức  $AX+BY=C$   
 Với cây nhị phân đã vẽ sẽ cho các biểu thức nào khi duyệt theo DLR và LDR ?



2) Hãy điền kết quả từng bước duyệt cây theo từng kiểu duyệt DLR, LDR, LRD vào trong bảng sau (giống như đã làm trong bài học):

| DLR      |        |      | LDR  |        |      | LRD  |        |      |
|----------|--------|------|------|--------|------|------|--------|------|
| Bước     | Đi qua | Hiện | Bước | Đi qua | Hiện | Bước | Đi qua | Hiện |
| 1        |        |      | 1    |        |      | 1    |        |      |
| 2        |        |      | 2    |        |      | 2    |        |      |
| ..       |        |      | ..   |        |      | ..   |        |      |
| ..       |        |      | ..   |        |      | ..   |        |      |
| ..       |        |      | ..   |        |      | ..   |        |      |
| ..       |        |      | ..   |        |      | ..   |        |      |
| ...      |        |      | ...  |        |      | ...  |        |      |
| ...      |        |      | ...  |        |      | ...  |        |      |
| ..       |        |      | ..   |        |      | ..   |        |      |
| ...      |        |      | ...  |        |      | ...  |        |      |
|          |        |      |      |        |      |      |        |      |
| Kết quả: |        |      |      |        |      |      |        |      |